

PRELIMINARY PROJECT REPORT

KopiBridge AI

Resume-to-AI-Tech-Role Gap Analyser

Student name: [Your Name]

Module: Final Year Project

Project template: Software Development / Web Application template

Date: 26 June 2026

Approximate word count: 4,300 words excluding references

Chapter 1: Introduction

Approximate word count: 720 words. Chapter maximum: 1,000 words.

KopiBridge AI is a resume-to-AI-tech-role gap analyser for final-year computer science students, fresh graduates, and early-career developers who want to apply for AI-adjacent technical roles. The project focuses on roles such as Junior AI Engineer, AI Solutions Engineer, LLM Application Developer, Software Engineer for AI Products, Technical Solutions Engineer, and Junior Data Analyst or Data Scientist. These roles often combine software engineering, data skills, communication, deployment, and emerging AI tooling. For students, the difficulty is not only learning these skills, but understanding whether their current resume proves the skills clearly enough for a target job description.

The motivation for this project comes from a common employability problem. Many students read AI job descriptions and see a long list of tools, frameworks, and responsibilities. They may then respond by adding generic keywords to their resume or asking a chatbot for broad advice. This can produce weak results because the advice may be vague, unrealistic, or even encourage the student to claim skills without evidence. KopiBridge AI takes a different position: a useful career tool should not simply make a resume sound better. It should help the applicant identify what is already supported, what is weak, and what proof they should build before making stronger claims.

The core project idea is therefore: "The local engine decides what is true. The LLM explains it better." In the current Stage 1 prototype, only the first part has been implemented. The system runs fully without an OpenAI API key or any external AI service. A user can upload a PDF resume, review the extracted text, paste a target job description, and receive a structured local analysis. The output includes an overall match score, score breakdown, role fit summary, matched requirements, missing or weak requirements, an evidence strength map, priority actions, resume improvements, and a 30-day improvement roadmap. The design intentionally avoids database accounts, authentication, RAG, and LLM routes at this stage so that the main analysis flow remains reliable and easy to demonstrate.

This project follows the Software Development / Web Application project template. The main contribution is a designed and evaluated web prototype, not a new machine learning model. The technical work involves PDF extraction in the browser, local text analysis, scoring logic, explainable evidence labels, UI design, print-ready report generation, and a feasible plan for future LLM wording polish. The project is still related to AI and employability because it targets AI-role preparation and may later use LLMs to improve explanation quality, but its Stage 1 value is based on deterministic, inspectable software behaviour.

The main aim of KopiBridge AI is to help early-career applicants turn a resume and target job description into an honest, practical improvement roadmap. The project objectives are:

1. To build a local-first web application that extracts or accepts resume text and compares it with a pasted AI-tech job description.
2. To design an explainable matching engine that identifies supported, partial, weak, and missing evidence for role requirements.
3. To generate honest recommendations that distinguish between "Safe to add" and "Needs proof first."

4. To present results in a polished report interface that can be understood quickly and saved as a PDF.
5. To evaluate the feasibility and usefulness of the feature prototype using software testing, usability criteria, and task-based review rather than inappropriate machine learning accuracy claims.

The intended users benefit because the tool translates job-market complexity into clear next steps. Instead of only saying "learn Docker" or "improve cloud skills," KopiBridge AI recommends proof-building actions, such as containerising one existing project, adding setup instructions, or deploying a working demo before adding the claim to a resume. This makes the output more ethical and practical for students, because the tool encourages evidence rather than exaggeration.

The scope of this preliminary report is the Stage 1 prototype. The LLM polishing route, API key handling, user accounts, and database-backed history are deliberately out of scope. The preliminary feature prototype demonstrates the most important technical feature: a reliable local analysis flow from resume and job description input to a structured career report. Later project stages can improve natural language quality and evaluation depth, but the current prototype already demonstrates the feasibility of the core system.

Chapter 2: Literature Review

Approximate word count: 1,600 words. Chapter maximum: 2,500 words.

2.1 Automated recruitment and applicant support

Automated recruitment systems are increasingly used to screen, rank, and manage job applications. Bogen and Rieke (2018) argue that hiring algorithms can make decisions more efficient, but they can also introduce opacity, bias, and accountability problems. Their work is important for this project because KopiBridge AI is not designed as an employer-side screening tool. It is an applicant-side support tool. The ethical risk is still relevant, however, because any system that interprets a resume can influence what a user believes about their employability.

Raghavan et al. (2020) also show that bias mitigation in algorithmic hiring is difficult because vendors may make claims about fairness without enough transparency into how systems actually work. This supports one of the design decisions in KopiBridge AI: the Stage 1 engine should be local, inspectable, and conservative. Instead of presenting an unexplained "AI judgement," the prototype shows matched evidence, missing evidence, score breakdowns, and honesty labels. This does not remove all risk, because even keyword-based matching can reflect the assumptions of its designer, but it reduces the danger of hiding the reasoning behind a black-box interface.

Most resume tools available to students focus on formatting, keyword optimisation, or generic advice. This can help with presentation, but it may also encourage applicants to optimise for applicant tracking systems without improving the underlying evidence in their projects. KopiBridge AI positions the resume as evidence of capability, not just as a keyword document. This aligns the tool with employability development rather than simple resume decoration.

2.2 Skills, occupations, and job matching

Skills taxonomies such as ESCO and O*NET provide structured ways to describe occupations, skills, knowledge, and work activities. ESCO is a European classification of skills, competences, qualifications, and occupations, while O*NET provides occupational information including skills, abilities, tasks, and work contexts. These systems show that employability matching is more complex than a single list of keywords. A job requirement may relate to tools, education, technical skills, experience, communication, or domain context.

KopiBridge AI uses a simplified internal skill signal taxonomy rather than a full external ontology. The prototype groups signals into categories such as skills, experience, tools, education, and communication, then applies category weights. This is less comprehensive than ESCO or O*NET, but it is appropriate for a focused Stage 1 prototype. The project targets AI-adjacent junior roles rather than the entire labour market, so a smaller curated signal set is easier to inspect, explain, and evaluate.

Natural language processing research shows that semantic matching can go beyond exact keywords. BERT introduced contextual language representations that improved many NLP tasks by modelling words in context (Devlin et al., 2019). Sentence-BERT adapted transformer models to produce sentence embeddings that can compare sentence meaning more efficiently (Reimers and Gurevych, 2019). These

methods are relevant because future versions of KopiBridge AI could use embeddings or LLMs to detect skill evidence even when the resume and job description use different wording.

However, semantic models also introduce trade-offs. They may improve recall, but they can make it harder to explain why a requirement was marked as matched. For a student-facing career tool, explainability is more important than appearing intelligent. A false positive, such as telling a student that they have RAG experience when they only mention "research," could lead to dishonest resume claims. For this reason, the current prototype uses conservative local matching based on curated aliases and related terms. This may miss some nuanced evidence, but the reasoning is visible and easier to correct.

2.3 Explainability, trust, and honest recommendations

Explainable AI research argues that users need understandable reasons for automated outputs, especially when those outputs affect decisions. Guidotti et al. (2018) survey methods for explaining black-box models and show that explanations can take different forms, including feature importance, examples, rules, and surrogate models. Doshi-Velez and Kim (2017) argue that interpretability should be evaluated in relation to the task and user, not treated as an abstract property.

These ideas strongly influence KopiBridge AI. The prototype does not only output a score. It presents an evidence map that labels each requirement as strong, partial, weak, or missing, and it provides short explanations. For example, if Docker appears in the job description but not in the resume, the tool should not say "Add Docker to your resume." It should say "Needs proof first: containerise one existing project before claiming Docker experience." This distinction matters because the tool is shaping user behaviour. The recommendation should help the student become more employable without encouraging misrepresentation.

The "Safe to add" and "Needs proof first" labels are a central design contribution. "Safe to add" is used when the resume already contains supporting evidence and the recommendation is mainly about clarity or wording. "Needs proof first" is used when a job requirement is missing or weakly evidenced. This creates a more ethical pattern than typical resume advice because it separates presentation improvements from capability-building actions.

Trust is also affected by reliability. A system that depends on a remote LLM route can fail during a demo, produce inconsistent explanations, or require paid API access. The Stage 1 prototype avoids this by making the local engine the source of truth. LLM polishing can be added later as an optional layer, but it should not decide the analysis. This architecture supports both practical reliability and user trust.

2.4 Career guidance and action planning

Gap analysis is only useful if the user can act on it. A weak tool may identify that a student lacks cloud, testing, or deployment evidence, but then provide vague advice such as "learn cloud" or "study testing." That advice is difficult for a student to convert into a portfolio improvement. KopiBridge AI therefore emphasises proof-building actions. The roadmap structure is fixed across four weeks: resume fixes and skill review, small proof project or feature, deployment/documentation/testing evidence, and resume rewrite/application preparation.

This approach suits early-career applicants because it connects career advice to visible artefacts. A student cannot always gain professional experience within 30 days, but they can improve evidence

quality by documenting an existing project, adding one implemented test, deploying a demo, writing a better README, or rewriting a project bullet around problem, action, technology, and result. The roadmap does not replace career counselling, but it provides a practical bridge between job requirements and project work.

There is a risk that action plans become too rigid. Different students have different backgrounds, time constraints, and target roles. The current prototype addresses this partly by basing roadmap tasks on the highest-priority gaps detected from the job description and resume. Future evaluation should test whether users find the roadmap specific enough, whether they can complete the tasks, and whether the tasks improve resume evidence in a later re-analysis.

2.5 Web usability and evaluation

Because KopiBridge AI is a web development project, its evaluation should not be framed as if it were a trained ML classifier. The current prototype does not claim statistical prediction accuracy. A more appropriate evaluation combines software correctness, usability, clarity, and perceived usefulness. ISO 9241-210 emphasises human-centred design for interactive systems, including understanding users, tasks, and context of use. Nielsen's usability heuristics also provide practical criteria such as visibility of system status, match between system and real-world language, error prevention, recognition rather than recall, and aesthetic minimalist design.

The System Usability Scale (SUS) introduced by Brooke (1996) is also relevant because it provides a lightweight way to collect user perceptions after task completion. For KopiBridge AI, an evaluation could ask participants to complete the core flow, interpret their match score, identify their top gaps, and explain what action they would take next. Measures could include task success, time on task, observed errors, confidence ratings, SUS score, and qualitative comments about whether the recommendations feel honest and actionable.

The literature therefore supports the project direction. Existing recruitment technologies show the value and risk of automated resume interpretation. Skill taxonomies show the need for structured matching. NLP research shows possible future semantic improvements, while explainability literature shows why transparent evidence is essential. Usability literature supports evaluating the prototype through real user tasks and report comprehension. KopiBridge AI combines these ideas in a focused local-first prototype for student career development.

Chapter 3: Design

Approximate word count: 1,250 words. Chapter maximum: 2,000 words.

3.1 Users and scenario

The primary user is a final-year computer science student or recent graduate applying for an AI-adjacent technical role. This user may have coursework, personal projects, internships, or hackathon work, but may not know how well those experiences map to a job description. The user may also be unsure which missing skills should be learned first. The system is designed for a laptop-based demo and real student workflow: upload resume, paste job description, analyse, review report, and save the output as a PDF.

A typical scenario is:

1. The student finds a Junior AI Engineer job description.
2. The student uploads their resume as a PDF.
3. The system extracts text and lets the student review or edit it.
4. The student pastes the job description.
5. The system produces a match report.
6. The student uses the 30-day roadmap to improve projects and resume evidence.

3.2 Functional and non-functional requirements

The main functional requirements are:

1. The user can upload a PDF resume or use sample resume text.
2. The system extracts readable PDF text in the browser where possible.
3. The user can paste or load a demo job description.
4. The system analyses the resume against the job description locally.
5. The result includes score, role summary, matched requirements, weak requirements, evidence map, priority actions, resume improvements, and roadmap.
6. The user can save the report by opening the browser print dialog.

The non-functional requirements are equally important. The app must work without an API key, database, authentication, or internet-dependent AI route. It should be responsive, readable, and stable during a live demonstration. It should not expose private resume data to a server in Stage 1. It should also avoid hallucinated recommendations by using deterministic local logic.

3.3 System architecture

The Stage 1 architecture is a client-side Next.js application using React, TypeScript, Tailwind CSS, and PDF.js. The local analysis engine is implemented as pure TypeScript functions. The main analysis function is:

```
analyseResumeAgainstJob(resumeText: string, jobDescription: string): AnalysisResult
```

The data flow is:

Resume PDF or sample text -> editable resume text -> job description text -> local analysis engine -> structured analysis result -> dashboard/report -> print/PDF save.

This architecture was chosen to reduce demo risk and preserve privacy. No resume data needs to be stored in a database. No account is required. No LLM is required to generate the report. A future Stage 2 route may polish selected wording, but it should only operate after the local AnalysisResult already exists. The LLM must not become the source of truth.

3.4 Local analysis design

The analysis engine uses a curated set of skill signals for AI-adjacent junior roles. Each signal has an id, label, category, aliases, related terms, recommended action, and weight. Examples include Python programming, machine learning fundamentals, LLM application development, RAG or embeddings workflow, API and backend development, testing, cloud platforms, Docker, deployment, Git, education, and stakeholder communication.

The engine first normalises the job description and selects relevant signals based on direct or related term hits. If too few signals are found, a default role-relevant set is added so that the report remains useful for broad job descriptions. Each selected signal is then scored against the resume text. Direct mentions and related terms produce an evidence strength:

1. Strong: multiple direct mentions or a direct mention with several supporting terms.
2. Partial: at least one direct mention, but limited supporting context.
3. Weak: only adjacent terms are found.
4. Missing: no direct or adjacent evidence is found.

Category scores are calculated as weighted averages of requirement-level evidence scores, and the overall score is calculated from category weights. The current category weights are skills 0.35, experience 0.26, tools 0.20, education 0.12, and communication 0.07. This weighting reflects the project assumption that technical skills and applied experience matter most for junior AI-adjacent roles, while education and communication still contribute.

Gap severity is derived from evidence strength. Missing requirements are high severity, weak requirements are medium severity, and partial requirements are treated as lower risk. The system then builds priority actions from the highest-ranked gaps. Each action uses proof-first wording so that the user knows what to build before making a resume claim.

3.5 Interface and report design

The interface is designed as a guided career report tool rather than a chatbot. The homepage clearly communicates the flow: Upload Resume -> Analyse Gap -> Get Roadmap. The input area is split into three steps: resume upload, job description input, and analysis. Secondary buttons allow a reliable demo path using sample resume text and a demo job description.

The results page is arranged around the information a viewer should understand within ten seconds: overall match score, role fit summary, top three priority gaps, top three recommended actions, and 30-day roadmap. Deeper sections such as evidence map, score breakdown, matched requirements, missing requirements, and resume improvements appear lower on the page. This prevents the report from feeling like a dense admin dashboard.

The visual design uses a refined coffee-inspired palette: warm cream backgrounds, white report cards, espresso text, tan borders, coffee-brown accents, muted green for strong evidence, amber for partial evidence, and clay red for high-priority gaps. The design goal is calm professionalism rather than decoration. The print stylesheet hides inputs and buttons, keeps the report content visible, uses white background and dark text, and avoids awkward card breaks where possible.

3.6 Workplan

The project workplan is feasible because the most technically risky Stage 1 flow has already been prototyped.

Stage 1: Local prototype and report interface. This stage includes PDF text extraction, editable resume text, job description input, local analysis, report dashboard, sample data, and print/save report. This has been implemented as the current feature prototype.

Stage 2: Evaluation and refinement. The next stage should run structured usability tests with students. Tasks should include loading sample data, interpreting the match score, identifying top gaps, and explaining which roadmap task they would complete first. Feedback should be used to improve wording, visual hierarchy, and edge cases.

Stage 3: Optional LLM polish. After local correctness and usability are stable, an optional server-side LLM route can improve phrasing of summaries and recommendations. It should not change scores, evidence labels, or gap severity. This preserves the local engine as the source of truth.

Stage 4: Final validation and project submission. The final stage should include cross-browser testing, mobile review, print/PDF testing, a demonstration video, final report writing, and a clear discussion of limitations.

3.7 Ethical and design considerations

KopiBridge AI handles sensitive resume data, so the Stage 1 local-only design is ethically useful. It reduces unnecessary data exposure and makes the prototype easier to explain. The tool also avoids telling users to add unsupported skills. This is important because employability tools can unintentionally encourage exaggeration. By using "Safe to add" and "Needs proof first," the design supports honest self-improvement.

The main limitation is that the matching logic is still simplified. It may miss valid evidence that is worded differently, and it may overvalue repeated terms. These limitations will be discussed openly in the evaluation and final report. The goal is not to claim perfect matching accuracy, but to demonstrate a useful, explainable, and reliable prototype.

Chapter 4: Feature Prototype

Approximate word count: 780 words. Chapter maximum: 1,500 words.

4.1 Prototype overview

The implemented feature prototype is the local explainable gap analysis flow. This was chosen because it is the most important technical feature in the project. If this feature works, the project can produce value even without LLM support. If it fails, later LLM polishing would only hide weak analysis. The prototype therefore demonstrates the feasibility of the core architecture.

The prototype is a Next.js web application. It includes PDF resume upload, browser-based PDF text extraction using PDF.js, editable extracted resume text, a job description text area, sample resume fallback, demo job description loading, local analysis, a polished report dashboard, and a Save Report button that calls `window.print()`.

4.2 Implemented behaviour

The main prototype flow is:

1. Open the app.
2. Click "Use Sample Resume Text" or upload a text-based PDF resume.
3. Click "Load Demo Job Description" or paste a real job description.
4. Click "Analyse Gap."
5. Review the generated dashboard.
6. Click "Save Report" to open the browser print dialog.

The generated report includes the required Stage 1 outputs: overall match score, score breakdown, role fit summary, matched requirements, missing or weak requirements, evidence strength map, top priority actions, recommended resume improvements, and a 30-day roadmap.

The analysis result is generated by local TypeScript functions. The engine normalises the resume and job description, selects relevant skill signals, counts direct and related term evidence, assigns evidence strength labels, calculates weighted category scores, ranks gaps by severity, and generates proof-building actions. The result is a typed `AnalysisResult`, which makes the report structure predictable and easier to test.

4.3 Technical challenge

The prototype is technically challenging enough for the preliminary stage because it combines several concerns that must work together reliably. PDF extraction has to run in the browser and fail gracefully when a PDF cannot be read. Text analysis must produce useful results without a remote AI service. The scoring logic must be consistent and explainable. The UI must organise a large amount of information

without overwhelming the user. The print report must preserve the important findings while hiding interactive input controls.

The project also has an important architectural constraint: it must work without `OPENAI_API_KEY`. This constraint forced the prototype to solve the core analysis problem locally instead of delegating the hard part to an LLM. As a result, the current version is more reliable for demonstrations and easier to evaluate.

4.4 Prototype evaluation

Because this is a web application prototype rather than a trained ML model, the evaluation approach focuses on feasibility, usability, reliability, and report usefulness. The current prototype has been checked against the intended demo path: opening the app, loading sample resume text, loading the demo job description, analysing the gap, viewing score/gaps/actions/roadmap/evidence map, and clicking Save Report. The Stage 1 verification commands also passed: lint, typecheck, and production build.

The prototype can be evaluated more formally with task-based usability testing. Participants should be final-year students or early-career applicants. Each participant can be asked to complete the core flow using either their own resume or sample data. The evaluation should record whether they can understand the overall score, identify the top three gaps, identify the top three recommended actions, explain the 30-day roadmap, and save the report as a PDF. After the task, a short SUS questionnaire and semi-structured questions can capture perceived ease of use and usefulness.

The evaluation should also include expert or heuristic review. Nielsen's usability heuristics are suitable because the tool needs clear system status, readable language, error prevention, and minimalist design. For example, PDF extraction errors should not crash the app, and recommendation labels should use real-world language such as "Needs proof first" rather than technical scoring jargon.

4.5 Evaluation of current results

The prototype works well as a Stage 1 feasibility demonstration. It proves that a resume and job description can be converted into a structured local career report without LLM dependency. The report is understandable because it prioritises score, fit summary, top gaps, actions, and roadmap before deeper evidence details. The Save Report function makes the output feel more like a career report than a temporary webpage.

The strongest part of the prototype is the honesty of the recommendations. It does not tell the user to add missing skills directly. Instead, it recommends proof-building tasks such as adding tests, deploying a project, documenting Docker setup, or creating a small RAG demo only when that gap is relevant. This supports the project aim of helping students improve evidence rather than exaggerate.

The main weakness is that the local matching engine is still rule-based. It can miss evidence if a resume uses unusual wording, and it cannot fully understand project context. For example, a student may have backend experience without using the exact word "API." Conversely, repeated keywords may not always prove deep competence. These limitations are acceptable for the preliminary prototype, but they should be addressed in later work through user testing, improved signal design, and optional LLM or embedding-based wording support.

4.6 Planned improvements

The next improvements should be controlled and evidence-based. First, the skill signal list should be reviewed against real AI-adjacent job descriptions and student resumes. Second, user testing should identify confusing labels, missing sections, and roadmap tasks that feel unrealistic. Third, the optional LLM stage should be added only for explanation polishing, not scoring. Fourth, the final evaluation should compare the original resume report with a revised resume after the user completes selected roadmap tasks. This would test whether KopiBridge AI actually helps students produce clearer evidence for target roles.

Overall, the feature prototype is successful for the preliminary stage. It demonstrates the project's central technical idea, supports a reliable demo, and provides a clear foundation for the final project.

References

- Bogen, M. and Rieke, A. (2018) Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias. Upturn. Available at: <https://www.upturn.org/work/help-wanted/>
- Brooke, J. (1996) 'SUS: A quick and dirty usability scale', in Jordan, P.W., Thomas, B., Weerdmeester, B.A. and McClelland, I.L. (eds.) Usability Evaluation in Industry. London: Taylor and Francis.
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', Proceedings of NAACL-HLT 2019, pp. 4171-4186. Available at: <https://aclanthology.org/N19-1423/>
- Doshi-Velez, F. and Kim, B. (2017) 'Towards a rigorous science of interpretable machine learning'. Available at: <https://arxiv.org/abs/1702.08608>
- European Commission (n.d.) What is ESCO? Available at: <https://esco.ec.europa.eu/en/about-esco/what-esco>
- Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F. and Pedreschi, D. (2018) 'A survey of methods for explaining black box models', ACM Computing Surveys, 51(5), Article 93. Available at: <https://doi.org/10.1145/3236009>
- ISO (2019) ISO 9241-210:2019 Ergonomics of human-system interaction - Part 210: Human-centred design for interactive systems. Available at: <https://www.iso.org/standard/77520.html>
- Mozilla (n.d.) PDF.js. Available at: <https://mozilla.github.io/pdf.js/>
- National Center for O*NET Development (n.d.) The O*NET Content Model. Available at: <https://www.onetcenter.org/content.html>
- Nielsen, J. (1994) 10 Usability Heuristics for User Interface Design. Nielsen Norman Group. Available at: <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Raghavan, M., Barocas, S., Kleinberg, J. and Levy, K. (2020) 'Mitigating bias in algorithmic hiring: Evaluating claims and practices', Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, pp. 469-481. Available at: <https://doi.org/10.1145/3351095.3372828>
- Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence embeddings using Siamese BERT-networks', Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. Available at: <https://aclanthology.org/D19-1410/>